

Contents

1. Introduction	3
Environment variables	3
User Variables:.....	3
System Variables:.....	3
Where to Set Environment Variables	3
Important Java Environment Variables	3
JAVA_HOME.....	3
PATH	3
CLASSPATH.....	3
Examples	4
2. Core OOP Principles (Pillars of OOP)	5
1. Class	6
2. Object.....	7
3. Abstraction	8
4. Encapsulation.....	9
5. Inheritance.....	10
6. Polymorphism.....	11
Compile-time Polymorphism (Method Overloading)	11
Runtime Polymorphism (Method Overriding).....	12
3. Java-Specific Implementation of OOP	13
4. Inheritance in Java	13
5. Polymorphism in Java	13
6. Abstraction in Java.....	14
7. Encapsulation in Java	14
8. Advanced OOP Features	14
9. OOP Design Concepts (For Deeper Understanding).....	14
Questions	15
Questions on Class.....	15
Basic Conceptual Questions on Java Class.....	15
Class Members (Fields and Methods)	16
Types and Modifiers	18
Class vs Interface	19
Code-based & Practical Questions	20
Memory and Design	25

Advanced	28
Questions on Object	29
Basic Conceptual Questions	29
Syntax and Usage.....	29
Object Behavior & Life Cycle	30
Code-based / Practical Questions.....	30
Advanced Object Usage.....	30
Other Questions	32

1. Introduction

Environment variables

Environment variables are settings in your operating system that affect how processes (like Java programs) run. They help Java tools (like javac, java) find the required files and libraries. There are two types:

User Variables:

- Specific to the currently logged-in user.
- Set by the user, not shared with other accounts.
- **Example:** You set JAVA_HOME only for your account.

System Variables:

- Global – apply to all users on the system.
- Needs administrator privileges to modify.
- **Example:** PATH, which helps the system locate Java tools like java, javac.

Where to Set Environment Variables

User Variables:

Control Panel → System → Advanced system settings → Environment Variables → "User variables"

System Variables:

Same place, under "System variables"

Important Java Environment Variables

JAVA_HOME

- Points to the Java installation directory.
- Used by tools like Maven, Gradle, Eclipse, IntelliJ, etc.
- **Example:** JAVA_HOME = C:\Program Files\Java\jdk-21

PATH

- Tells the system where to look for executable files like java and javac.
- You add %JAVA_HOME%\bin to PATH so you can run Java commands in any folder.
- **Example:** PATH = ...;C:\Program Files\Java\jdk-21\bin

CLASSPATH

- Tells the Java compiler (javac) and JVM (java) where to find user-defined classes and packages.
- Not always needed. It's used when your Java program depends on External .jar files (like MySQL Connector, Apache POI, etc.), Custom class locations.
- **Example:** CLASSPATH = .;C:\MyJavaLibs\mysql-connector.jar (. means current directory is included)

Examples

Example 01

Running Java Without Setting PATH

```
> javac Hello.java
```

'javac' is not recognized as an internal or external command.

After adding Java's bin directory to PATH, it works:

```
> javac Hello.java
```

```
> java Hello
```

Example 02

Using a JAR Library

You have:

mysql-connector.jar

Your program: DBConnect.java

Set CLASSPATH temporarily:

```
> set CLASSPATH=.;C:\MyLibs\mysql-connector.jar
```

```
> javac DBConnect.java
```

```
> java DBConnect
```

2. Core OOP Principles (Pillars of OOP)

- Class
- Object
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
 - Compile-time (Method Overloading)
 - Runtime (Method Overriding)

1. Class

Definition: A class is a **blueprint or template** for creating objects. It defines the **properties (fields)** and **behaviors (methods)** that the objects created from the class will have.

Key Points:

- Defines **structure and behavior**
- Can have fields, methods, constructors, and blocks
- No memory allocated until an object is created

```
class Car {  
  
    String color;  
  
    int speed;  
  
    void drive() {  
        System.out.println("Car is driving");  
    }  
}
```

2. Object

Definition: An object is an **instance of a class**. It occupies memory and represents a real-world entity.

We create objects in Java to **bring classes to life**, access their members, and build structured, modular, and maintainable programs. Without objects, Java would not support **object-oriented programming**.

Key Points:

- Created using `new` keyword
- Has state (fields) and behavior (methods)
- You can create multiple objects from one class

```
Car myCar = new Car();  
myCar.color = "Red";  
myCar.drive();
```

3. Abstraction

Definition: Abstraction means **hiding the internal implementation** and showing only the **essential features** to the user.

```
abstract class Animal {  
    abstract void sound(); // Abstract method  
    abstract void eat(); // Abstract method  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
    void eat(){  
        System.out.println("Meat");  
    }  
}
```

Key Points:

- Achieved using **abstract classes** and **interfaces**
- Helps manage complexity
- Promotes loose coupling

4. Encapsulation

Definition: Encapsulation is the **wrapping of data (variables)** and **code (methods)** together into a **single unit (class)**. It also restricts direct access to data using **access modifiers**.

```
class Person {  
    private String name; // Private variable  
  
    public String getName() { // Public getter  
        return name;  
    }  
  
    public void setName(String name) { // Public setter  
        this.name = name;  
    }  
}
```

Key Points:

- Achieved using **private variables** and **public getters/setters**
- Provides **data hiding**
- Makes the code more secure and maintainable

5. Inheritance

Definition: Inheritance is the process where one class **acquires the properties and methods** of another class.

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks.");  
    }  
}  
  
class Monkey extends Animal {  
    void jump() {  
        System.out.println("Monkey Jumps.");  
    }  
}
```

Key Points:

- `extends` keyword is used
- Promotes **code reuse**
- Java supports **single**, **multilevel**, and **hierarchical** inheritance (but not multiple inheritance with classes)

6. Polymorphism

Definition: Polymorphism means "**many forms**". It allows methods to behave **differently** based on the object that invokes them.

Compile-time Polymorphism (Method Overloading)

Definition: Same method name, but different **parameter lists** in the same class.

```
class MathUtils {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

Key Points:

- Resolved at **compile time**
- Improves readability
- Also called **static polymorphism**

Runtime Polymorphism (Method Overriding)

Definition: A **subclass provides a specific implementation** of a method that is already defined in its superclass.

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Cat extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Meow");  
    }  
}
```

```
Animal a = new Cat(); // Runtime polymorphism  
a.sound(); // Output: Meow
```

Key Points:

- Uses **method overriding**
- Achieved via **inheritance**
- Resolved at **runtime**
- Enables **dynamic method dispatch**

3. Java-Specific Implementation of OOP

- Classes and Objects in Java
 - Constructors (Default, Parameterized, Copy)
 - `this` keyword
 - `new` keyword
 - `static` keyword (with variables, methods, blocks)
 - `final` keyword (variable, method, class)
 - Access Modifiers (`private`, `public`, `protected`, `default`)
-

4. Inheritance in Java

- Single Inheritance
 - Multilevel Inheritance
 - Hierarchical Inheritance
 - Why Java doesn't support Multiple Inheritance with classes
 - `super` keyword (constructor and method usage)
-

5. Polymorphism in Java

- **Compile-time (Static Polymorphism)**
 - Method Overloading
 - **Runtime (Dynamic Polymorphism)**
 - Method Overriding
 - Dynamic Method Dispatch
 - `instanceof` operator
-

6. Abstraction in Java

- Abstract Classes and Methods
 - Interfaces
 - Default methods (Java 8+)
 - Static methods
 - Functional interfaces (Java 8+)
 - Difference between abstract class vs interface
-

7. Encapsulation in Java

- Getters and Setters
 - JavaBeans convention
 - Data hiding and access control
-

8. Advanced OOP Features

- Nested Classes (Static and Non-static inner classes)
 - Anonymous Classes
 - Lambda Expressions (Java 8+)
 - Object class methods (`toString()`, `equals()`, `hashCode()`, `clone()`)
 - `enum` in Java
 - Record Classes (Java 14+)
-

9. OOP Design Concepts (For Deeper Understanding)

- Association, Aggregation, Composition
- Coupling and Cohesion
- SOLID Principles (for writing better OOP code)
- DRY, KISS, YAGNI (coding principles)
- Design Patterns (basic):
 - Singleton, Factory, Builder, etc.

Questions

Questions on Class

Basic Conceptual Questions on Java Class

1. What is a class in Java?

- A class in Java is a blueprint or template for creating objects.
- It defines properties (variables) and behaviors (methods) common to all objects of that type

2. How is a class different from an object?

- A class is a logical construct; it defines structure and behavior but does not hold data itself.
- An object is a real instance of a class with actual values stored in its fields.
- Class = Definition; Object = Instance created from that definition.

3. Why do we use classes in Java?

- To achieve object-oriented programming principles like encapsulation, inheritance, and polymorphism.
- To organize code better and make it reusable and modular.
- To represent real-world entities in software development.

4. What is the syntax of declaring a class in Java?

```
class ClassName {  
    // fields (variables)  
    // methods  
}  
Example:  
  
class Car {  
    String model;  
    void drive() {  
        System.out.println("Car is driving");  
    }  
}
```

5. What is the default access modifier for a class?

- The default access modifier is package-private (no explicit modifier).
- This means the class is accessible only within classes in the same package.

6. Can a Java class be declared inside another class? What is it called?

- Yes, a class can be declared inside another class in Java.
- It is called a nested class.
- Nested classes can be:
 - Static nested class
 - Inner class (non-static)

7. What is the difference between a class and a structure (struct in C/C++)?

- Memory Allocation: Class objects are usually stored in heap memory; struct instances may be stack-allocated (in C/C++).
- Access Control: Classes support access modifiers (private, protected, public); structs in C/C++ do not have access control (everything is public by default).
- Inheritance: Java classes support inheritance; C/C++ structs do not (in C) or have limited support (in C++).
- Methods: Java classes can contain both data (fields) and methods; C structs only contain data.
- Object-Oriented Support: Classes are fundamental to OOP in Java; structs are primarily used for grouping data in C/C++.

Class Members (Fields and Methods)

8. What are class variables and instance variables?

- Class Variables:
Declared using the static keyword inside a class.
Shared by all instances (objects) of the class.
One copy exists for the entire class.
Example: static int totalCars;
- Instance Variables:
Declared without static.
Each object has its own copy.
Represents the state of a specific object.
Example: String color;

9. What are static and non-static members in a class?

- Static Members:
Belong to the class itself, not to any object.
Shared across all objects.
Can be accessed using the class name.
Includes static variables and static methods.
- Non-Static Members:
Belong to specific objects (instances) of the class.
Each object has its own copy.
Requires object creation to access.

10. How are methods defined inside a class?

- Methods are defined inside the class using the following syntax:

```
returnType methodName(parameters) {  
    // method body  
}  
  
class Car {  
    void drive() {  
        System.out.println("Car is driving");  
    }  
}
```

Methods can be static or non-static.

11. What is a constructor? How is it different from a method?

- Constructor:
A special block of code used to initialize objects.
Has the same name as the class.
No return type (not even void).
Automatically called when an object is created.
- Difference from Method:
Methods perform actions; constructors initialize objects.
Methods can have any name; constructors must have the class name.
Methods must specify a return type; constructors do not.

12. Can a class have multiple constructors? What is constructor overloading?

- Yes, a class can have multiple constructors.
- This is called constructor overloading.
- Constructor overloading means defining multiple constructors with different parameter lists.
- It allows creating objects in different ways.

Example:

```
class Car {  
    Car() {  
        // Default constructor  
    }  
  
    Car(String model) {  
        // Parameterized constructor  
    }  
}
```

Types and Modifiers

13. What are different types of classes in Java? (e.g., regular class, abstract class, final class)

- **Regular (Concrete) Class:**
A standard class that can be instantiated to create objects.
- **Abstract Class:**
A class declared with the abstract keyword.
Cannot be instantiated directly; must be subclassed.
Can contain abstract methods (without implementation) and concrete methods (with implementation).
- **Final Class:**
A class declared with the final keyword.
Cannot be subclassed (i.e., cannot be extended).
- **Nested Class:**
A class defined within another class. (Detailed below.)
- **Enum Class:**
Represents a fixed set of constants. Defined using the enum keyword.

14. What is the use of the final keyword with a class?

When a class is declared as final:

- It cannot be subclassed (no other class can extend it).
- It is used to prevent inheritance, ensuring the class's implementation cannot be changed through extension.

```
final class Car {  
    // Class content  
}
```

15. What is an abstract class? Can we create an object of it?

- An abstract class is a class that is declared using the abstract keyword.
- It may contain:
Abstract methods (without body)
Concrete methods (with body)
- We cannot create an object of an abstract class directly.
- It is meant to be inherited by other classes that provide implementations for its abstract methods.

16. What is a nested class? List its types.

A nested class is a class defined within another class.

Types of Nested Classes in Java:

- **Static Nested Class:**
Declared with static keyword.
Does not have access to instance members of the outer class
- **Non-Static Nested Class (Inner Class):**

Tied to an instance of the outer class.

- **Local Inner Class:**
Defined within a block, usually inside a method.
- **Anonymous Inner Class:**
A class without a name, declared and instantiated in a single statement, usually used for short implementations.

17. Can a class be both abstract and final? Why or why not?

No, a class cannot be both abstract and final.

Reason: An abstract class is meant to be inherited (subclassed) to provide implementations for its abstract methods.

A final class cannot be inherited.

These two concepts contradict each other, so Java does not allow it.

Class vs Interface

18. What is the difference between a class and an interface?

Class: Represents a blueprint for creating objects. It can define attributes (fields) and methods (behavior).

Interface: Represents a contract or capability that a class can implement (like "Flyable", "Drawable"). It defines method signatures but no implementation (until Java 8+).

- Before Java 8: Only method signatures (no body).
- From Java 8: Can have default and static methods with bodies.
- From Java 9: Can also have private methods.

19. Can a class implement multiple interfaces?

Java allows multiple interface implementation to support multiple inheritance of type (since Java doesn't support multiple class inheritance directly)

20. Can a class extend more than one class? Why?

No, a class cannot extend more than one class in Java.

Because Java does not support multiple inheritance with classes to avoid ambiguity and complexity, especially the Diamond Problem.

Code-based & Practical Questions

21. Write a simple Java class with fields, constructors, and methods.

```
public class Car {

    // Fields (instance variables)
    String brand;
    String color;
    int year;

    // Default constructor
    public Car() {
        brand = "Unknown";
        color = "White";
        year = 2000;
    }

    // Parameterized constructor
    public Car(String brand, String color, int year) {
        this.brand = brand;
        this.color = color;
        this.year = year;
    }

    // Method to display car details
    public void displayInfo() {
        System.out.println("Car Brand: " + brand);
        System.out.println("Color: " + color);
        System.out.println("Year: " + year);
    }

    // Method to update the year
    public void updateYear(int newYear) {
        year = newYear;
    }

    // Main method to run the program
    public static void main(String[] args) {
        // Using default constructor
        Car car1 = new Car();
        car1.displayInfo();

        // Using parameterized constructor
        Car car2 = new Car("Toyota", "Red", 2022);
        car2.displayInfo();

        // Updating the year
        car2.updateYear(2025);
        car2.displayInfo();
    }
}
```

22. Write a program that creates objects of a class and calls its methods.

```
public class Student {
    // Fields
    String name;
    int age;
    String course;

    // Constructor
    public Student(String name, int age, String course) {
        this.name = name;
        this.age = age;
        this.course = course;
    }

    // Method to display student information
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
    }
}

public class Main {
    public static void main(String[] args) {
        // Create objects of Student class
        Student s1 = new Student("Alice", 20, "Computer Science");
        Student s2 = new Student("Bob", 22, "Mechanical Engineering");
        Student s3 = new Student("Charlie", 21, "Electronics");

        // Call methods on each object
        s1.displayInfo();
        s2.displayInfo();
        s3.displayInfo();
    }
}
```

23. Demonstrate the use of static variables and static methods inside a class.

```
public class Counter {  
    // Static variable (shared among all objects)  
    static int count = 0;  
  
    // Instance variable  
    String name;  
  
    // Constructor  
    public Counter(String name) {  
        this.name = name;  
        count++; // Increment static variable when a new object is created  
    }  
  
    // Static method  
    public static void showCount() {  
        System.out.println("Total objects created: " + count);  
    }  
  
    // Instance method  
    public void showName() {  
        System.out.println("Name: " + name);  
    }  
  
    // Main method to test  
    public static void main(String[] args) {  
        Counter c1 = new Counter("Object1");  
        Counter c2 = new Counter("Object2");  
        Counter c3 = new Counter("Object3");  
  
        c1.showName();  
        c2.showName();  
        c3.showName();  
  
        // Calling static method (recommended using class name)  
        Counter.showCount();  
    }  
}
```

24. Write a program that uses constructor overloading in a class.

```
public class Book {
    String title;
    String author;
    double price;

    // Constructor 1: No parameters (default)
    public Book() {
        title = "Unknown";
        author = "Unknown";
        price = 0.0;
    }

    // Constructor 2: Only title
    public Book(String title) {
        this.title = title;
        author = "Unknown";
        price = 0.0;
    }

    // Constructor 3: Title and author
    public Book(String title, String author) {
        this.title = title;
        this.author = author;
        price = 0.0;
    }

    // Constructor 4: Title, author, and price
    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }

    // Method to display book details
    public void displayInfo() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Price: ₹" + price);
    }

    // Main method to test
    public static void main(String[] args) {
        Book b1 = new Book();
        Book b2 = new Book("Java Basics");
        Book b3 = new Book("Effective Java", "Joshua Bloch");
        Book b4 = new Book("Clean Code", "Robert C. Martin", 499.99);

        b1.displayInfo();
        b2.displayInfo();
        b3.displayInfo();
        b4.displayInfo();
    }
}
```

25. Create a class Student with fields name, rollNo, and marks and print student details.

```
public class Student {  
    // Fields  
    String name;  
    int rollNo;  
    double marks;  
  
    // Constructor  
    public Student(String name, int rollNo, double marks) {  
        this.name = name;  
        this.rollNo = rollNo;  
        this.marks = marks;  
    }  
  
    // Method to display student details  
    public void displayDetails() {  
        System.out.println("Student Name: " + name);  
        System.out.println("Roll Number: " + rollNo);  
        System.out.println("Marks: " + marks);  
    }  
  
    // Main method to test the class  
    public static void main(String[] args) {  
        // Create a Student object  
        Student s1 = new Student("Alice", 101, 89.5);  
  
        // Call method to display details  
        s1.displayDetails();  
    }  
}
```


Memory and Design

26. How is a class loaded into memory in Java?

In Java, class loading is the process of **bringing a class into memory** so it can be used by the JVM (Java Virtual Machine). This is part of the **Java Runtime Environment** and happens **dynamically at runtime**.

Phases of Class Loading

Loading

- The .class file (bytecode) is loaded by the ClassLoader.
- It brings the class into memory.
- It creates a Class object for the class in the Method Area.

Linking

- This has 3 sub-steps:
- Verification: Bytecode is checked for correctness and security.
- Preparation: Static variables are created and assigned default values.
- Resolution: References to other classes are resolved (like method and field lookups).

Initialization

- Static variables are assigned their actual values.
- Static blocks (if any) are executed.
- This is the final step before the class is ready to be used.

```
class Example {
    static {
        System.out.println("Static block executed");
    }

    Example() {
        System.out.println("Constructor called");
    }

    public static void main(String[] args) {
        System.out.println("Main method started");
        Example obj = new Example(); // triggers class loading
    }
}
```

Static block executed

Main method started

Constructor called

27. What is a class loader?

A **Class Loader** in Java is a part of the **Java Runtime Environment (JRE)** that **dynamically loads classes into memory** when they are needed during execution.

Java programs don't load all classes at once — instead, **class loaders load classes on demand**, improving efficiency and modularity.

Role of Class Loader

- Reads the `.class` files (bytecode).
- Converts them into **Class objects** in memory.
- Ensures classes are **loaded only once** into the JVM.

◆ **Types of Class Loaders**

Class Loader	Description
1. Bootstrap Class Loader (Primordial)	Loads core Java classes from the JDK (like <code>java.lang</code> , <code>java.util</code>). Written in native code.
2. Extension Class Loader	Loads classes from the <code>ext</code> folder of the JDK (<code>lib/ext</code>).
3. Application (System) Class Loader	Loads classes from the CLASSPATH (user-defined classes).
4. Custom Class Loader	You can write your own by extending <code>ClassLoader</code> class. Useful for frameworks, security, or hot swapping.

28. What is the difference between a class and an object in terms of memory?

✓ Difference Between a **Class** and an **Object** in Terms of Memory

Aspect	Class	Object
Definition	A template or blueprint to create objects	A real instance of the class
Memory Allocation	No memory is allocated for fields directly	Memory is allocated when an object is created
Stored In	Stored in Method Area of JVM when loaded	Stored in Heap Memory
Static Members	Stored once with the class in Method Area	Each object has its own copy of non-static members
When It Exists	At compile-time and class loading time	At runtime , when <code>new</code> is used

Memory Visualization Example

java

```
class Student {  
    String name;      // Instance variable  
    int rollNo;  
    static String college = "ABC College"; // Static variable  
}
```

java

```
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student();  
    }  
}
```

Memory Behavior:

- `Student` class (structure + static variable `college`) → stored once in **Method Area**
- `s1` and `s2` → separate **objects** in **Heap**, each with their own `name` and `rollNo`
- `college` is shared between `s1` and `s2`, stored only once

Advanced

29. What is reflection? How can we get class information at runtime using reflection?

30. What is a singleton class? How do you implement it in Java?

31. What is a POJO (Plain Old Java Object)?

32. What is the difference between an inner class and a static nested class?

Questions on Object

Definition: An object is an **instance of a class**. It occupies memory and represents a real-world entity.

We create objects in Java to **bring classes to life**, access their members, and build structured, modular, and maintainable programs. Without objects, Java would not support **object-oriented programming**.

Key Points:

Created using `new` keyword

Has state (fields) and behavior (methods)

You can create multiple objects from one class

```
Car myCar = new Car();  
myCar.color = "Red";  
myCar.drive();
```

Basic Conceptual Questions

1. What is an object in Java?

2. How is an object different from a class?

3. What are the steps involved in creating an object in Java?

4. Can you create an object without using the `new` keyword? If yes, how?

5. What happens in memory when you create an object?

6. Why can't we call non-static methods without creating an object?

7. What is the default value of an object reference in Java?

Syntax and Usage

8. How do you create an object from a class in Java?

9. What is object instantiation?

10. Can we have multiple objects of the same class?

11. Can an object access both static and non-static members?

12. What is the purpose of constructors in object creation?

Object Behavior & Life Cycle

13. What is object reference?

14. What is the scope of an object?

15. When is an object eligible for garbage collection?

16. What is the difference between object creation and object initialization?

17. What happens if we try to use an object reference that is null?

Code-based / Practical Questions

18. Write a program to demonstrate creating multiple objects of a class.

19. Write a program to access class variables and methods through objects.

20. Demonstrate the difference between static and non-static members using objects.

21. Write a program to pass an object as a method parameter.

22. Write a program that returns an object from a method.

Advanced Object Usage

23. What is object cloning? How do you clone an object in Java?

24. What is the difference between shallow copy and deep copy of an object?

25. What is the use of the this keyword with objects?

26. What are anonymous objects? Give an example.

27. How does Java handle object comparison (using == vs .equals())?

Other Questions

28. Why Do We Need to Create an Object in Java?

A **class** is like a blueprint (e.g., blueprint of a car).
An **object** is the actual **instance** created from that blueprint (a real car built from the plan).
You need to create objects to:
Access **non-static** methods and variables.
Represent **real-world entities** like a `Student`, `BankAccount`, or `Car`.
Enable **modularity**, **reusability**, and **data encapsulation**.

```
class Car {
    String color = "Red";
    void drive() {
        System.out.println("The car is driving.");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // object created
        System.out.println(myCar.color); // access variable
        myCar.drive(); // call method
    }
}
```

29. Advantages of Creating Objects in Java

Advantage	Description
Encapsulation	Objects allow grouping data (variables) and behavior (methods) together, hiding internal details.
Reusability	Classes can be reused by creating multiple objects without rewriting code.
Modularity	Code is easier to manage because each object handles its own data and functions.
Flexibility	Objects make it easier to modify one part of the program without affecting others.
Real-world modeling	Objects let you model real-world entities in code more naturally.
Inheritance	You can create new classes based on existing ones, reusing and extending functionality.
Polymorphism	Objects of different classes can be treated as objects of a common superclass, allowing dynamic behavior.

